

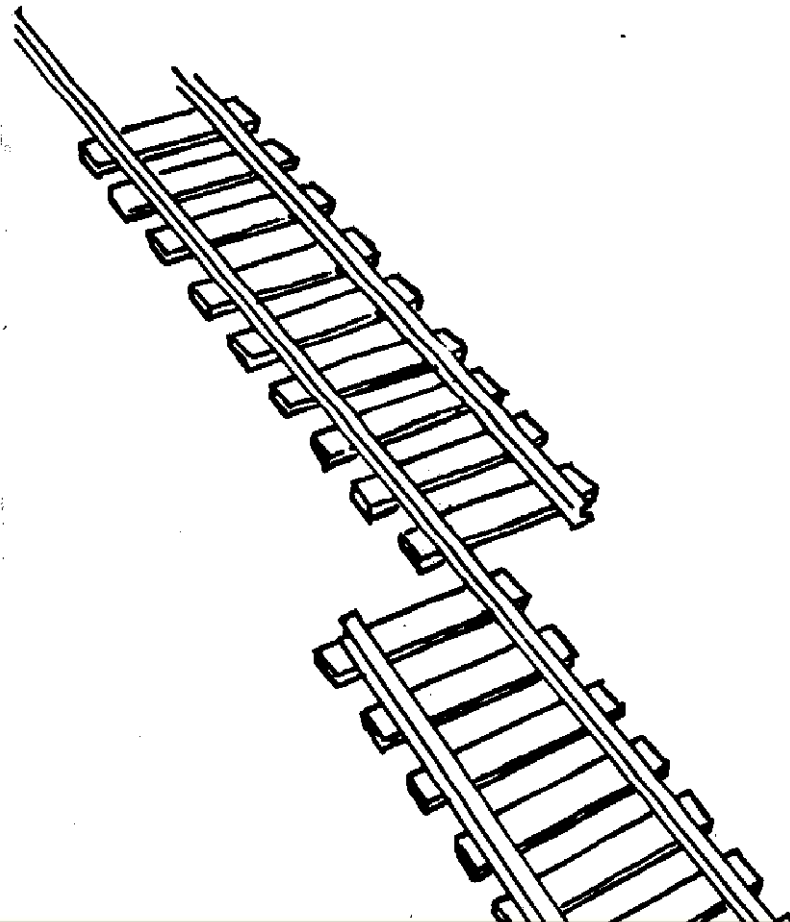
9. Testiranje

KAKO ugotovimo KJE smo naredili napake?

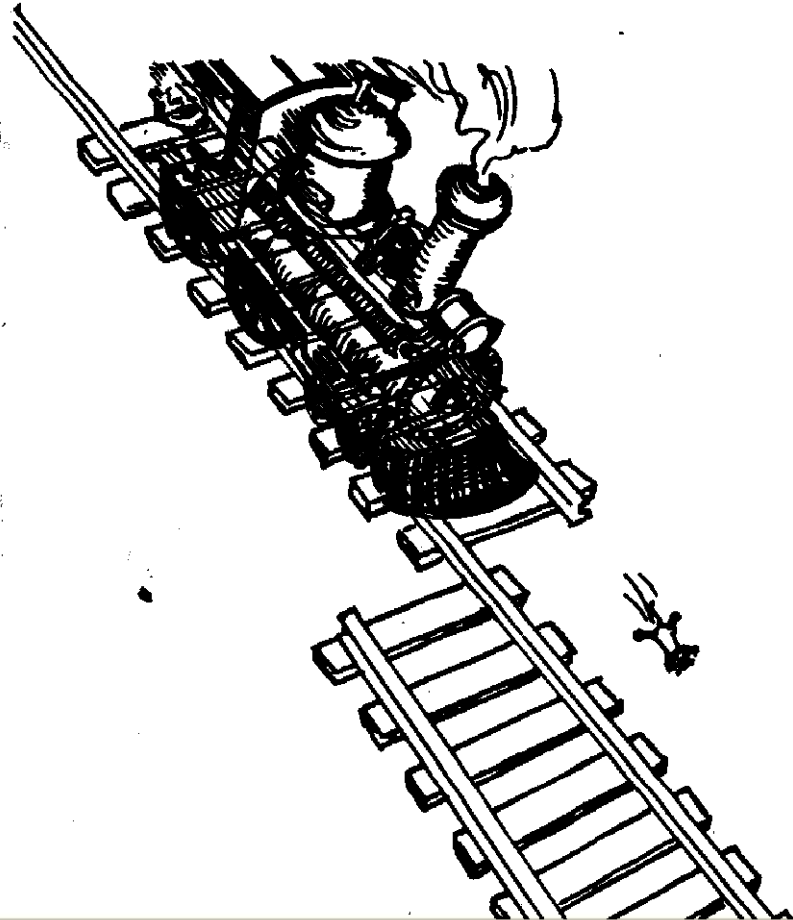
Terminologija

- **Zanesljivost** (reliability): Merilo ujemanja obnašanja sistema s specifikacijami obnašanja.
- **Odpoved** (failure): Vsak odklon opazovanega obnašanja od specificiranega obnašanja.
- **Napaka** (error): Sistem je v takšnem stanju, da ga bo nadaljnje izvajanje privedlo do odpovedi.
- **Napaka-hrošč** (bug): Mehanski ali algoritmični vzrok za napako.
- Obstaja mnogo različnih tipov napak in različnih načinov, kako se z napakami spoprimemo.

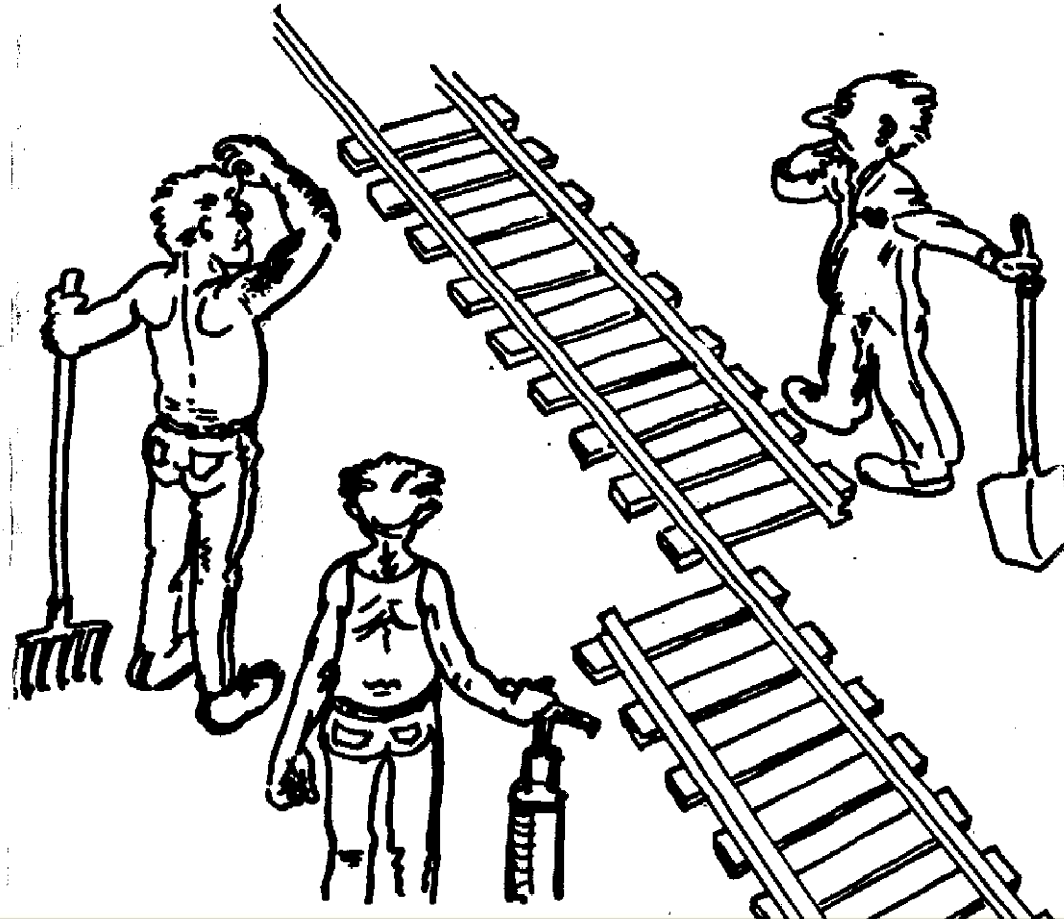
Kaj je to?



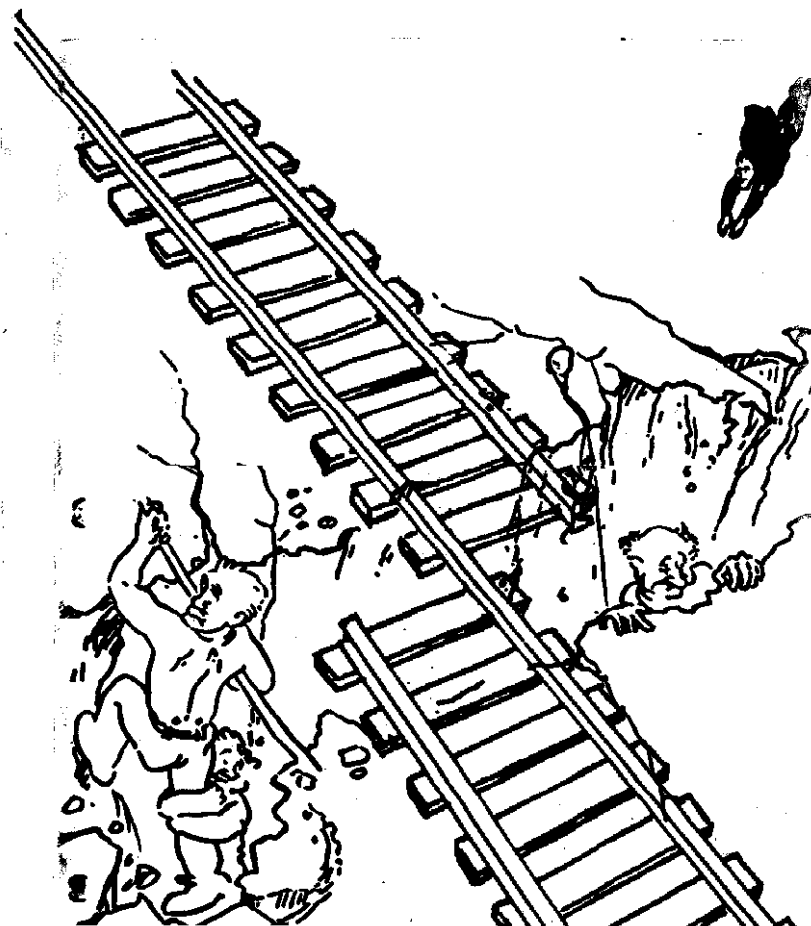
Stanje napake (“napaka”)



Algoritmična napaka

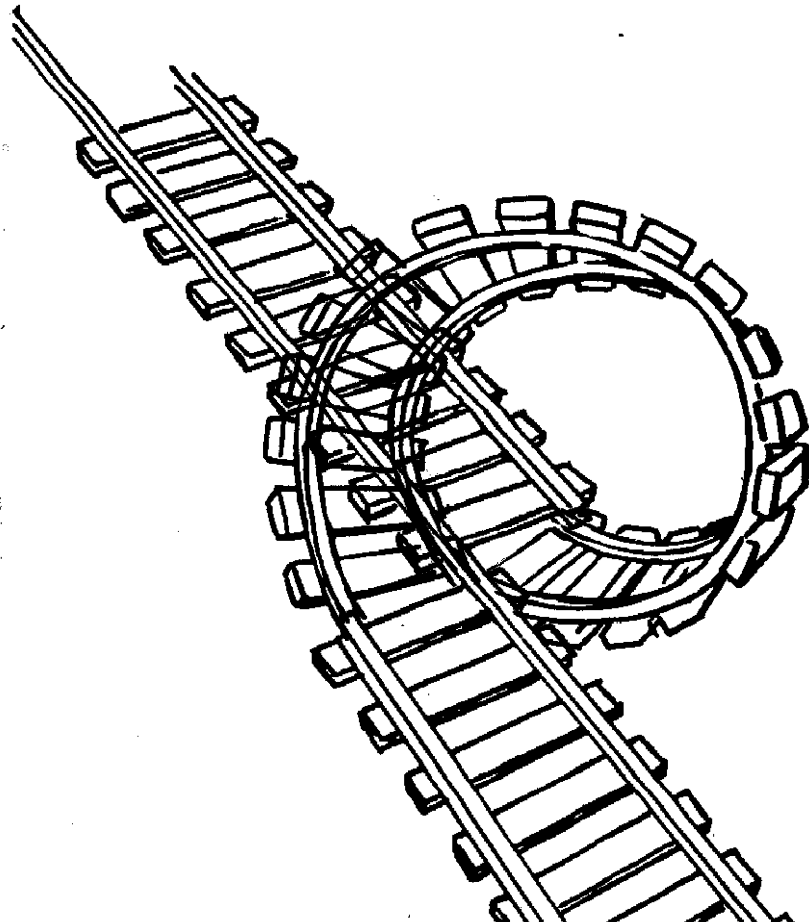


Mehanska napaka

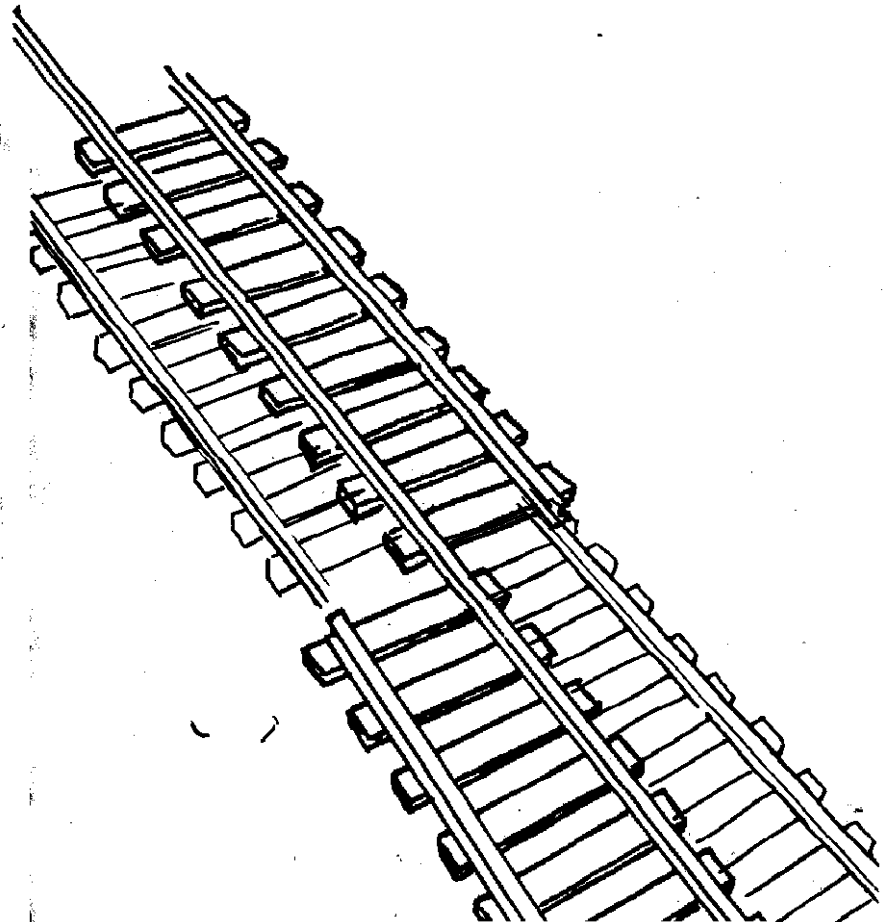


Kako se spoprimemo z napakami?

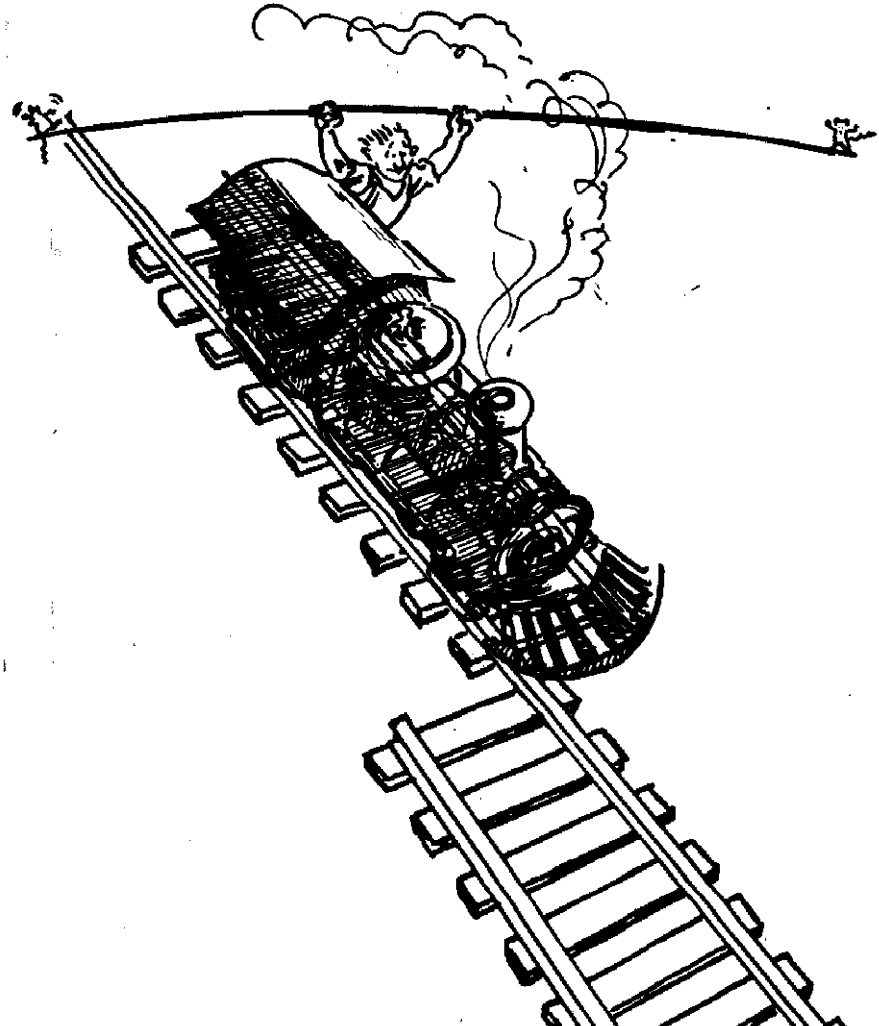
Verifikacija?



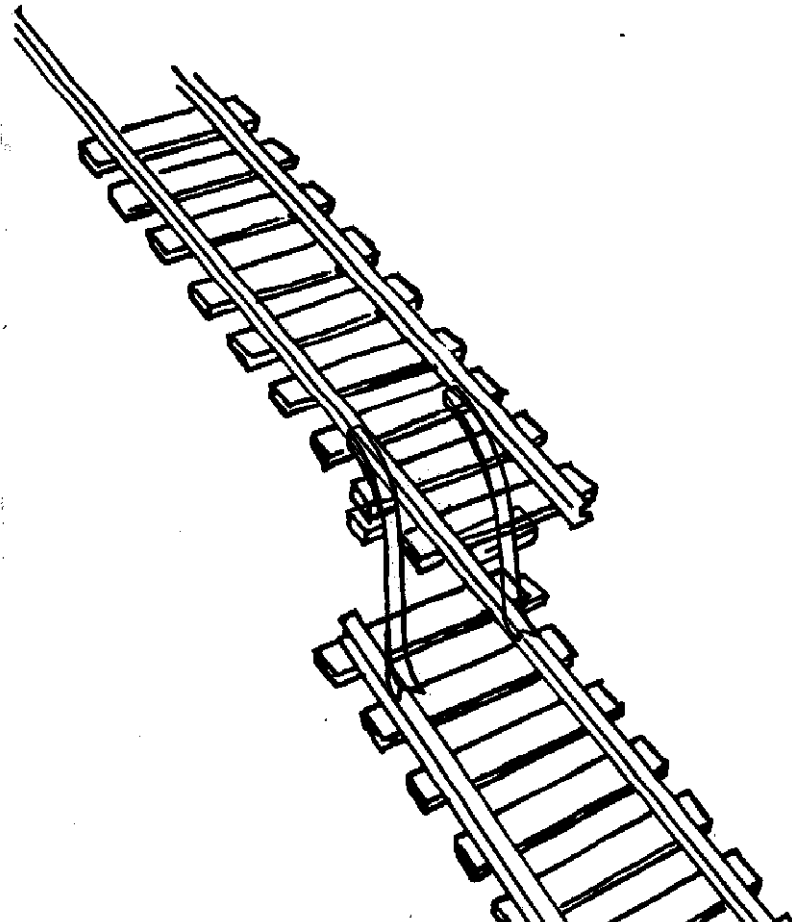
Modularna redundanca?



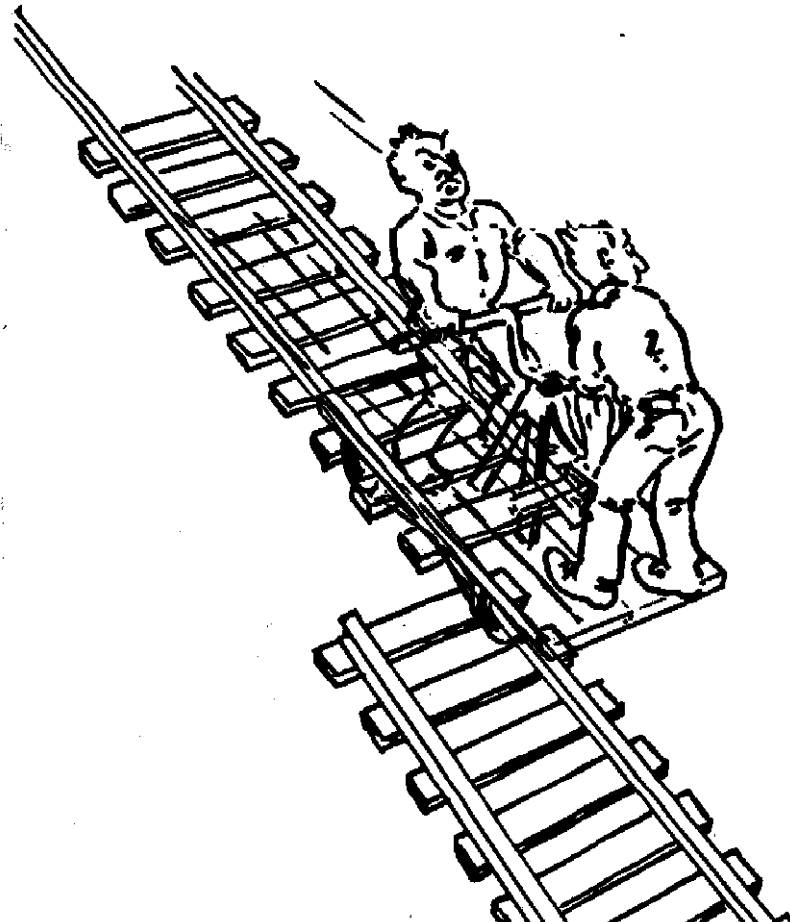
Razglasitev napake za lastnost?



Krpanje?



Testiranje?



Ravnanje z napakami

- Verifikacija (verification)
 - Predvideva hipotetično okolje, ki se dostikrat ne ujema z realnim okoljem
- Modularna redundanca (modular redundancy)
 - Drago!
- Razglasitev napake za lastnost sistema (declaring bug as a feature)
 - Slab pristop
- Krpanje (patching)
 - Pripelje k “edinstvenim” sistemom (one-of-a-kind systems)
- **Testiranje** (testing)
 - Testiranja ni nikdar preveč in običajno ni dovolj dobro opravljeno

Kako drugače ravnati z napakami

- Najbolje se je **izogibati napakam** (zmanjševanje možnosti za nastanek napak, preden je sistem izdan):
 - Uporabimo primerno **razvojno metodologijo**, ki zmanjša kompleksnost
 - Uporabimo **nadzor nad verzijami**, da preprečimo nekonsistentnost sistema
 - Uporabimo **verifikacijo**, da se izognemo algoritmičnim napakam oz. hroščem
- Žal se vsem napakam običajno ne moremo izogniti!

Kako drugače ravnati z napakami

- **Detekcija** (pred ali med izvajanjem sistema):
 - Testiranje: iščemo napake na planiran način
 - Razhroščevanje: začnemo z neplanirano odpovedjo
 - Nadzor (monitoring): spremljamo informacije o stanju sistema; poiščemo napake, ki vplivajo na učinkovitost
- **Toleranca** (**okrevanje po napaki**, ko je sistem že izdan):
 - Sistemi podatkovnih baz – razveljavljanje transakcij (rollback)
 - Modularna redundanca
 - Deli za okrevanje (recovery)

Nekatera spoznanja

- Nobenega ne-trivialnega modula ali sistema **ni mogoče popolnoma testirati**
 - Teoretične omejitve: kompleksnost naloge, vzpostavljanje pogojev delovanja
 - Praktične omejitve: omejen čas in omejeni stroški
- Testiranje lahko pokaže samo **prisotnost napak**, nikdar ne dokaže njihove odsotnosti (E.W. Dijkstra)

Primer: izčrpno testiranje

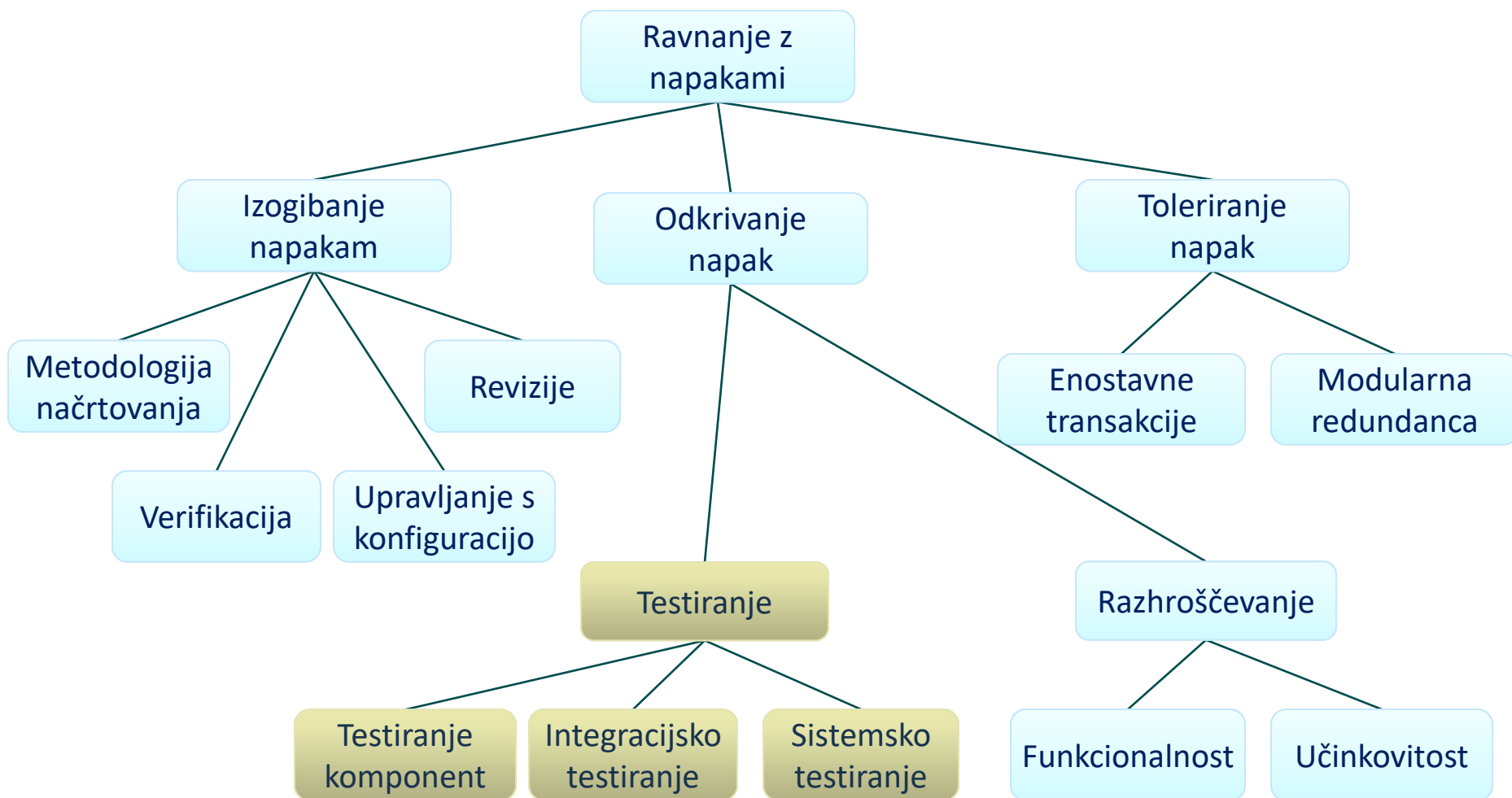


Možnih je 10^{14} poti!
Če izvedemo en test v
milisekundi, bi testiranje
tega programa trajalo 3.170
let!!

→ Nesprejemljivo!

zanka: 20x

Tehnike ravnanja z napakami



Definicija testiranja

“ Testiranje je proces poganjanja programa z namenom, da odkrijemo napake. ”

Glen Myers

Testabilnost = kako enostavno lahko program testiramo (1/2)

- Lastnosti, ki pomagajo oz. vplivajo na testiranje:
 - **Izvedljivost** – čisto izvajanje programa
 - Možnost **opazovanja**
 - Rezultate je enostavno videti
 - Za vsak vhod dobimo različne izhode
 - Napačne izhode je enostavno identificirati
 - Možnost **nadzora**
 - Procesiranje lahko nadziramo
 - Teste lahko avtomatiziramo in po-uporabimo

Testabilnost = kako enostavno lahko program testiramo (2/2)

- **Razgradljivost**
 - Posamezne dela sistema lahko testiramo neodvisno
- **Enostavnost**
 - Ni kompleksne arhitekture in programske logike
- **Stabilnost**
 - Med testiranjem je potrebnih le malo sprememb
- **Razumljivost**
 - Program zlahka razumemo

Kdo testira?



Razvijalci

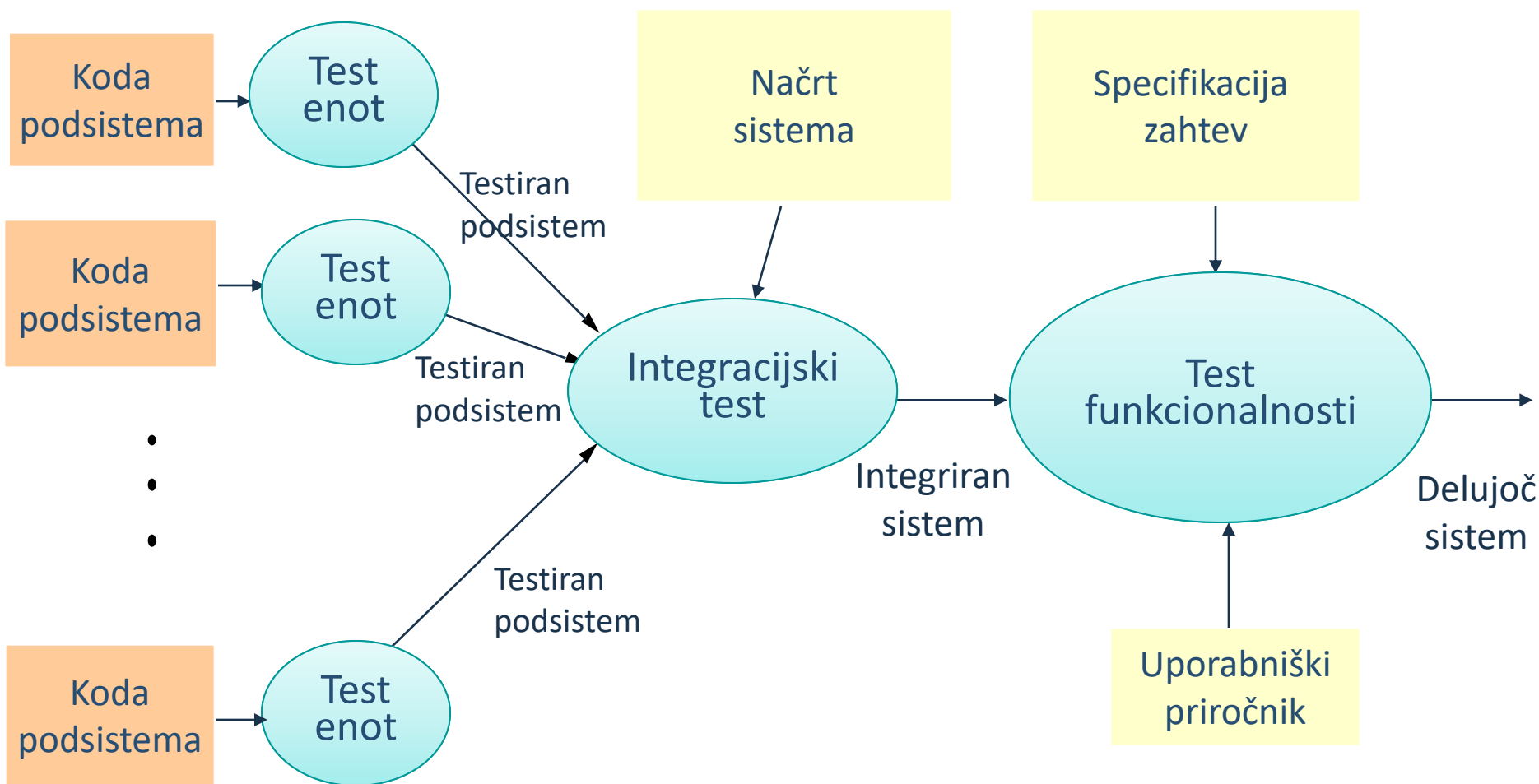
- poznajo in razumejo sistem
- “nežno” testiranje
- usmerjeni k dobavi sistema
- nekritični do sistema



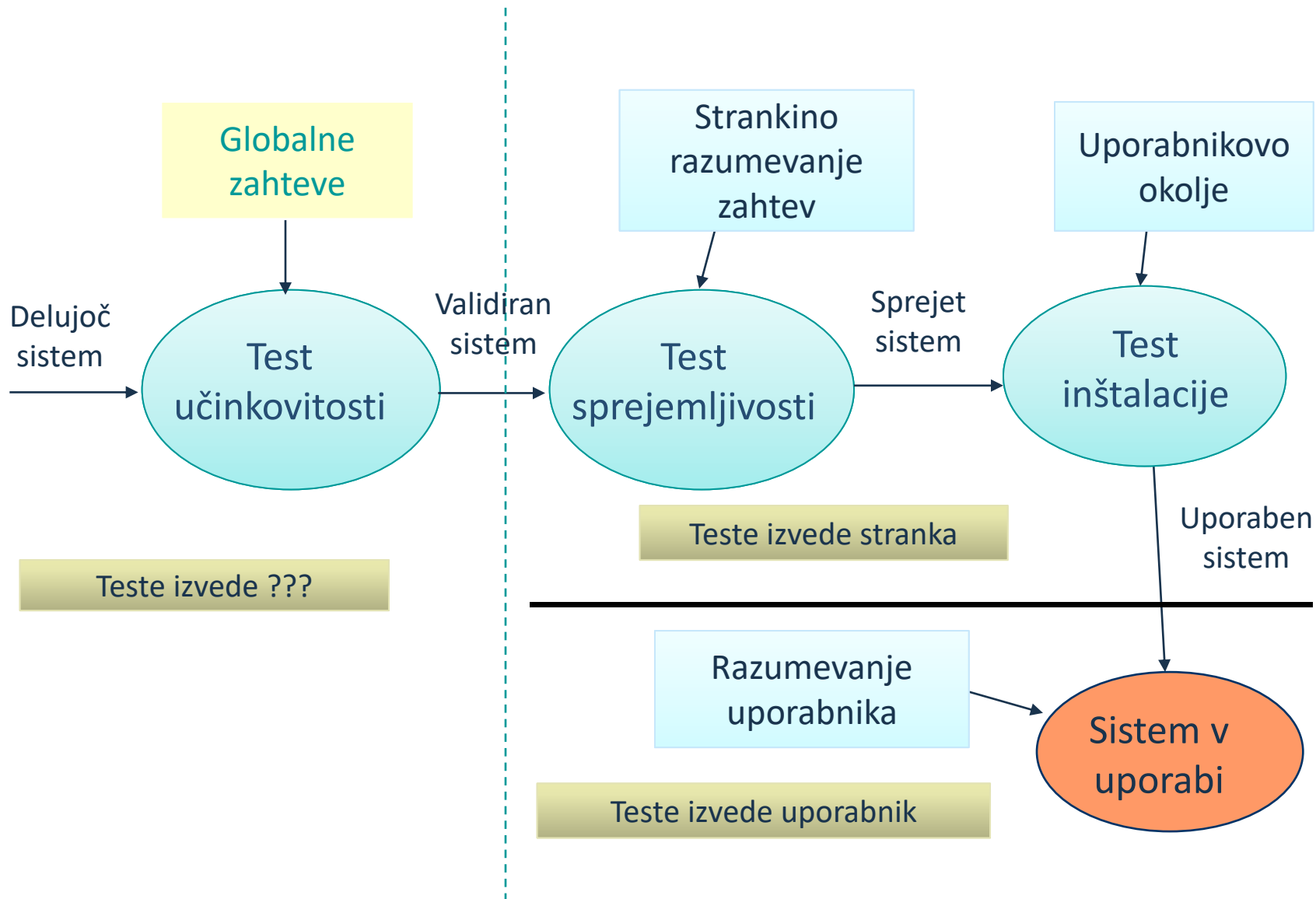
Neodvisni testerji

- sistem morajo šele spoznati
- poskušajo “zlomiti” sistem
- usmerjeni so h kakovosti
- kritični do sistema

Aktivnosti testiranja (1/2)



Aktivnosti testiranja (2/2)



Nivoji testiranja

- Testiranje komponent
 - **Testiranje enot** (unit testing)
- Integracijsko testiranje (integration testing)
 - Združevanje enot v celoto
- Sistemsko testiranje (system testing)
 - Sistem kot celota

Testiranje enot

- Testiranje enot (unit testing):
 - Kaj: Posamezni podsistemi (moduli oz. komponente, funkcije, ...)
 - Kdo: Testiranje izvede razvijalec
 - Cilj: Potrditev, da je enota pravilno implementirana in zagotavlja načrtovano funkcionalnost

Testiranje enot



Integracijsko testiranje

- Integracijsko testiranje (integration testing):
 - Kaj: Skupine (posamezno že testiranih) podsistemov oz. modulov
 - Kdo: Testiranje izvede razvijalec
 - Cilj: Testiranje povezav med posameznimi podsistemi in njihovo delovanje kot celota

Testiranje sistema

- ▣ Sistemsko testiranje (system testing):
 - ▣ Kaj: Celoten sistem
 - ▣ Kdo: Testiranje izvede razvijalec skupaj z naročnikom
 - ▣ Cilj: Ugotoviti, ali sistem izpolnjuje zahteve (funkcionalne in globalne)

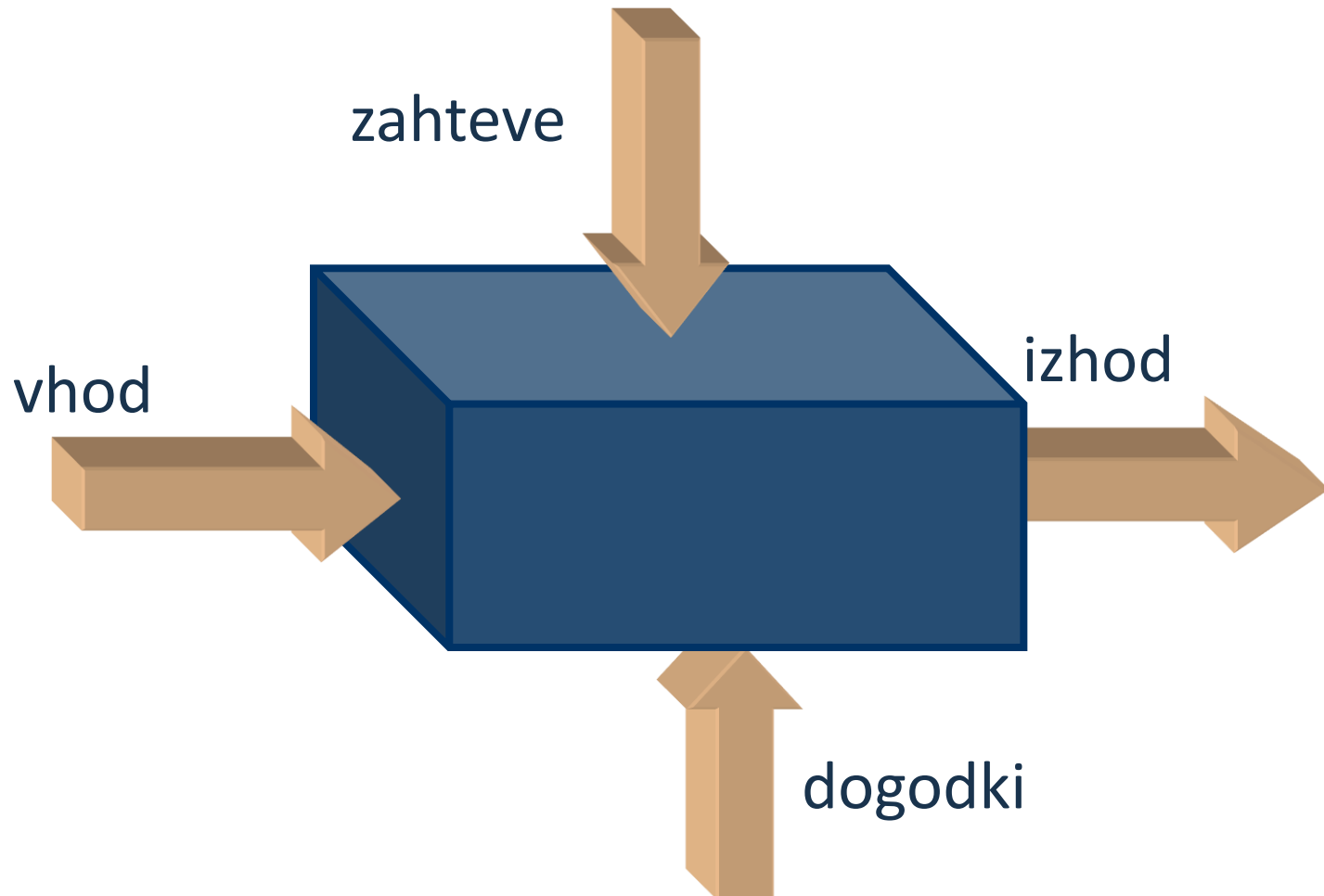
Pristopi k testiranju enot

- Neformalno testiranje
 - Inkrementalno kodiranje
- Statična analiza
 - **Ročno izvajanje**: branje izvornega koda
 - Sprehod skozi kod (koder neformalno prikaže izvorni kod ostalim)
 - Inšpekcija koda (koder formalno prikaže izvorni kod ostalim)
 - **Avtomatizirana orodja** za preverjanje:
 - Sintaktičnih in semantičnih napak
 - Neupoštevanje standardov kodiranja

Pristopi k testiranju enot

- Dinamična analiza
 - Testiranje z metodo **črne škatle** (black-box testing)
 - Testiranje vhodov/izhodov
 - Testiranje z metodo **bele škatle** (white-box testing)
 - Testiranje interne strukture oz. logike podsistema ali objekta
 - Testiranje na osnovi podatkovnih struktur
 - Podatkovni tipi določajo testne primere

Testiranje z metodo črne škatle



Testiranje z metodo črne škatle

- Fokus: **obnašanje vhodov/izhodov**. Če se za vsak podan vhod izhod ujema s predvidenim, potem enota opravi test.
 - V večini primerov je nemogoče preizkusiti vse možne vhode (testne primere)
- Cilj: zmanjšati število testnih primerov z ekvivalenčnim particioniranjem:
 - Razdelimo vhodne pogoje v ekvivalenčne razrede
 - Testne primere izberemo iz vsakega ekvivalenčnega razreda

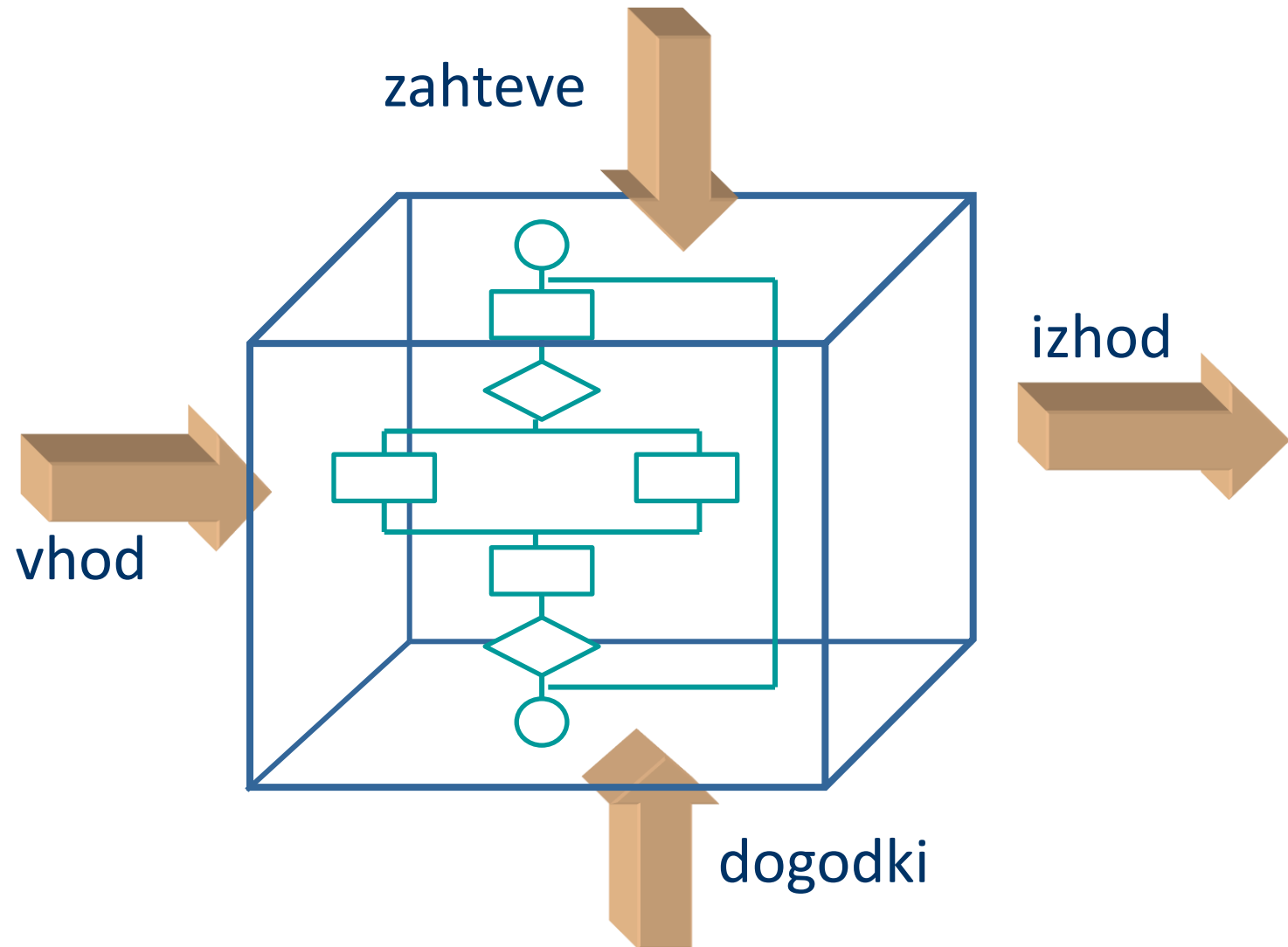
Izbira ekvivalenčnih razredov

- **Primer #1:** vhod je veljaven za nek interval vrednosti (npr. med 100 in 5000). Določimo tri ekvivalenčne razrede:
 - Pod intervalom ($x < 100$)
 - Znotraj intervala ($100 \leq x \leq 5000$)
 - Nad intervalom ($5000 < x$)
- **Primer #2:** vhod je veljaven za množico diskretnih vrednosti (npr. $\{A, B, C\}$). Določimo dva ekvivalenča razreda:
 - Veljavna diskretna vrednost ($x \in \{A, B, C\}$)
 - Neveljavna diskretna vrednost ($x \notin \{A, B, C\}$)

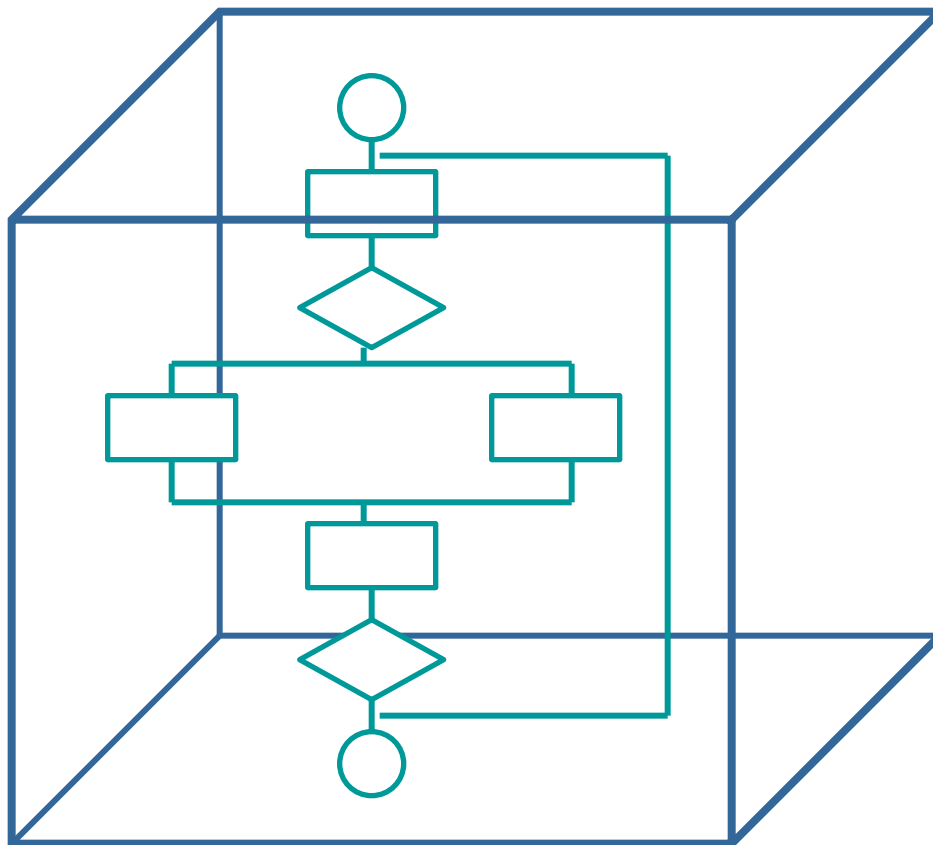
Kategorije napak pri testiranju s črno škatlo

- ❑ Nepravilne ali manjkajoče funkcije
- ❑ Napake v vmesnikih
- ❑ Napake v podatkovnih strukturah ali zunanjem dostopu do podatkovnih baz
- ❑ Neprimerna učinkovitost
- ❑ Napake v inicializaciji ali terminaciji

Testiranje z metodo bele škatle



Testiranje z metodo bele škatle



Naš namen je zagotoviti, da se bodo **vsi stavki in pogoji** izvedli vsaj enkrat.

Testiranje z metodo bele škatle

- Fokus: temeljitost (**pokritost**). Vsak stavek komponente izvedemo vsaj enkrat.
- Štirje tipi testiranja po metodi bele škatle:
 - Testiranje stavkov
 - Testiranje zank
 - Testiranje poti
 - Testiranje vejitev

Testiranje z metodo bele škatle

- Testiranje zank:
 - Določimo pogoje tako, da se **zanka ne izvede** (izjema so REPEAT oz. DO-WHILE zanke)
 - Izvedemo zanko **natančno enkrat**
 - Izvedemo zanko **več kot enkrat**
- Testiranje poti:
 - Poskrbimo, da se **izvedejo vse poti** v programu
- Testiranje vejitev (testiranje pogojev):
 - Zagotovimo, da je **vsak možen izhod iz pogoja** testiran vsaj enkrat

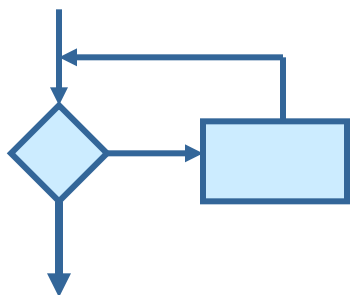
```
if (i == TRUE)
    printf("YES");
else
    printf("NO");
```

Testna primera:

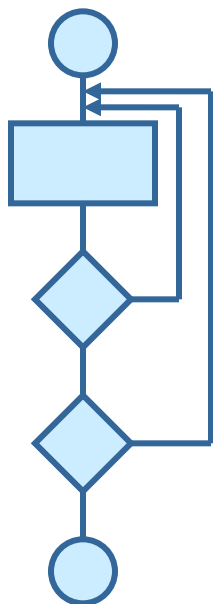
1. i=TRUE
2. i=FALSE

Testiranje zank

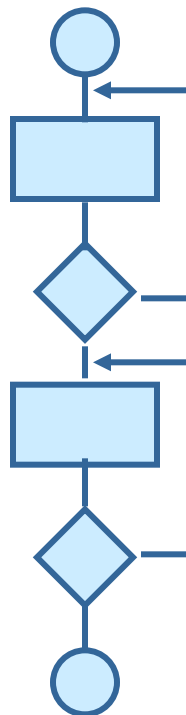
- Zanke lahko vodijo do “zazankanih” programov



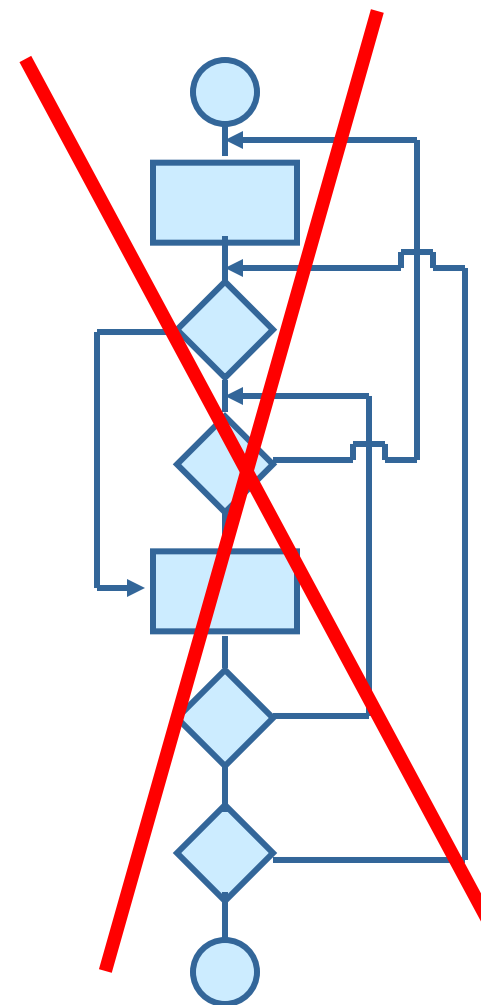
enostavne
zanke



gnezdene zanke



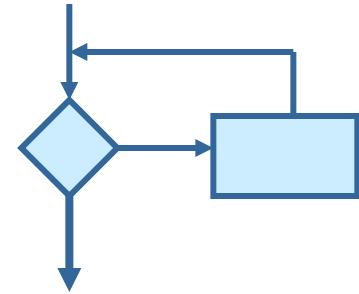
zaporedje zank



nestrukturirane
zanke

Testiranje zank: enostavne zanke

- Minimalni pogoji:
 - Zanko popolnoma preskočimo
 - Samo en prehod skozi zanko
 - m prehodov skozi zanko, $m < n$
 - $n-1, n, n+1$ prehodov skozi zanko



Primer: testiranje z belo škatlo

```
IzracunajPovprecje(FILE FTocke) {  
    float VsotaTock = 0.0; float SteviloOcen = 0;  
    float Povprecje = 0; float Ocena;  
    Read(FTocke, Ocena); // Beri in seštej točke  
    while (! EOF(FTocke) ) {  
        if ( Ocena > 0.0 ) {  
            VsotaTock = VsotaTock + Ocena;  
            SteviloOcen++;  
        }  
        Read(FTocke, Ocena);  
    }  
    /* Izračunaj povprečje in izpiši rezultat */  
    if (SteviloOcen > 0 ) {  
        Povprecje = VsotaTock/SteviloOcen;  
        printf("Povprečna ocena je %f \n", Povprecje);  
    } else  
        printf("V datoteki ni ocen.\n");  
}
```

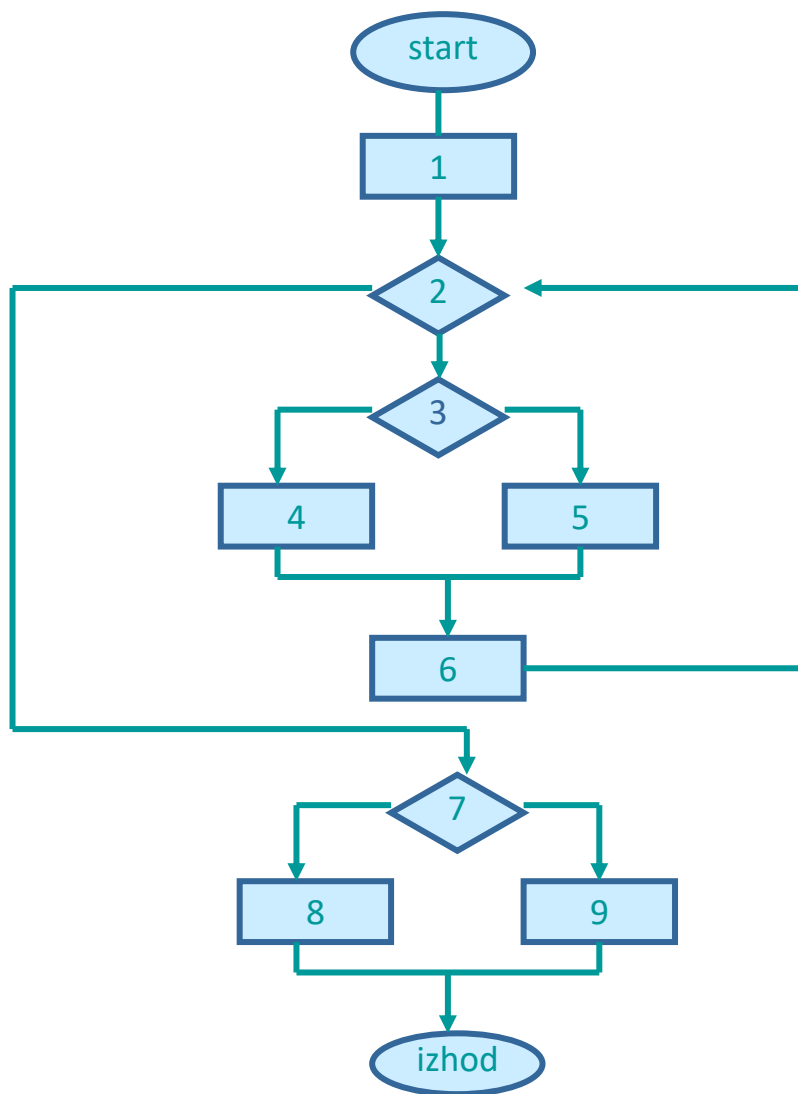
Primer: določitev poti

```
IzracunajPovprecje(FILE FTocke) {  
    float VsotaTock = 0.0; float SteviloOcen = 0;  
    float Povprecje = 0; float Ocena;  
    Read(FTocke, Ocena); // Beri in seštej točke  
    while (! EOF(FTocke) ) {  
        if ( Ocena > 0.0 ) {  
            VsotaTock = VsotaTock + Ocena;  
            SteviloOcen++;  
        }  
        Read(FTocke, Ocena);  
        /* Izračunaj povprečje in izpiši rezultat */  
        if (SteviloOcen > 0 ) {  
            Povprecje = VsotaTock/SteviloOcen;  
            printf("Povprečna ocena je %f \n", Povprecje);  
        } else {  
            printf("V datoteki ni ocen.\n");  
        }  
    }  
}
```

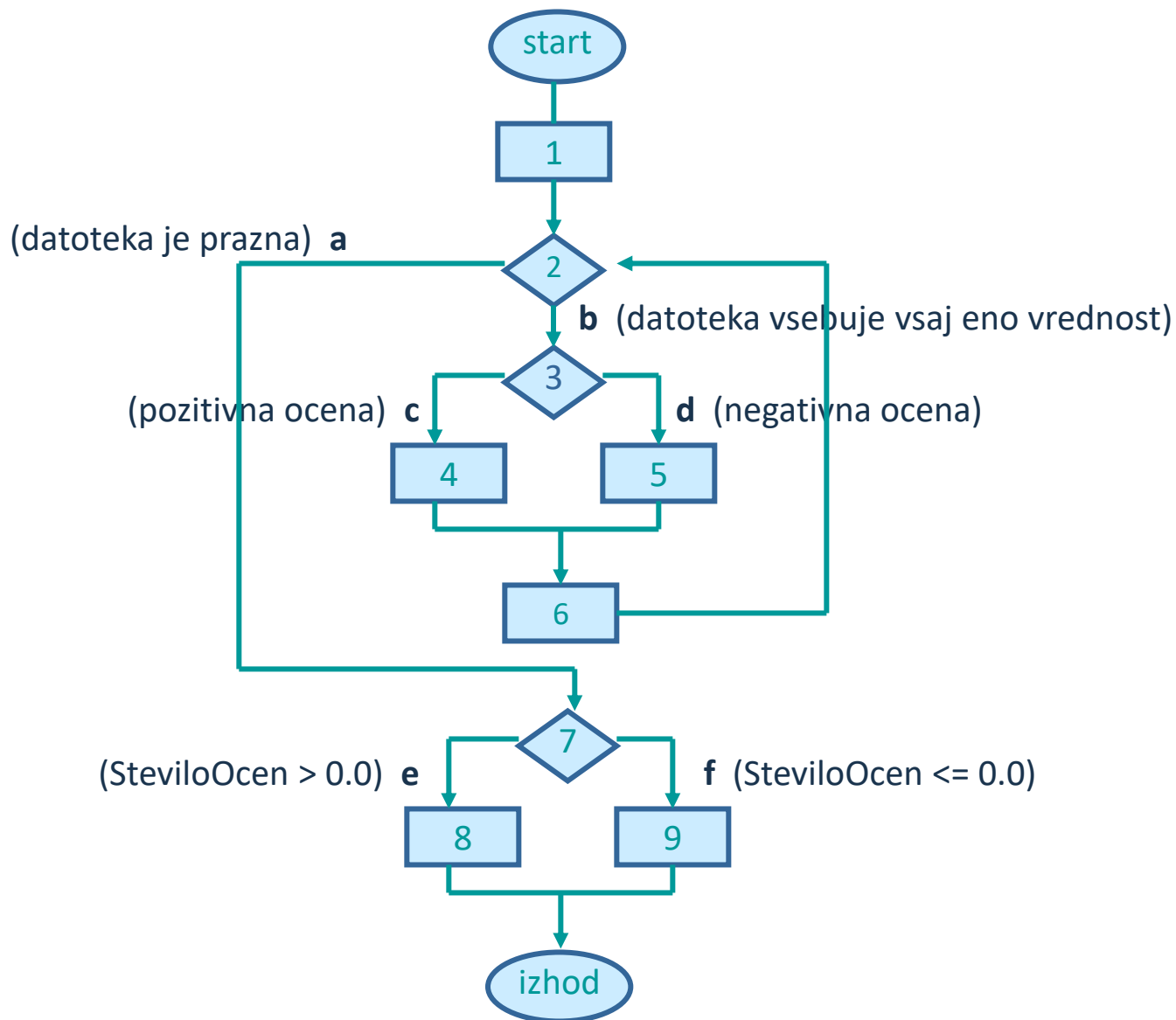
Diagram illustrating the execution flow of the `IzracunajPovprecje` function:

- Initialization of variables: `float VsotaTock = 0.0; float SteviloOcen = 0; float Povprecje = 0; float Ocena;` and reading the first value: `Read(FTocke, Ocena);` (labeled 1).
- Start of the `while` loop: `while (! EOF(FTocke) ) {` (labeled 2).
- Start of the `if` statement: `if ( Ocena > 0.0 ) {` (labeled 3).
- Calculation of the sum and increment of the counter: `VsotaTock = VsotaTock + Ocena;` and `SteviloOcen++;` (labeled 4).
- End of the `if` statement: `}` (labeled 5).
- Reading the next value: `Read(FTocke, Ocena);` (labeled 6).
- Start of the `if` statement for the average calculation: `if (SteviloOcen > 0 ) {` (labeled 7).
- Calculation of the average and printing the result: `Povprecje = VsotaTock/SteviloOcen;` and `printf("Povprečna ocena je %f \n", Povprecje);` (labeled 8).
- Printing the message if no scores were found: `printf("V datoteki ni ocen.\n");` (labeled 9).

Primer: diagram logičnih tokov



Primer: izbiranje testnih primerov



Primerjava: črna vs. bela škatla (1/2)

- Testiranje z metodo bele škatle:
 - Testirati moramo potencialno neskončno število poti
 - Z belo škatlo pogosto testiramo kaj **JE** narejeno, ne pa kaj **NAJ BI BILO** narejeno!
 - Ne moremo najti izpuščenih primerov uporabe
- Testiranje z metodo črne škatle:
 - Možna kombinatorična eksplozija testnih primerov (veljavnih in neveljavnih podatkov)
 - Pogosto ni jasno, ali izbrani testni primeri razkrijejo določeno napako
 - Ne odkrijejo nekoristnih primerov uporabe (“lastnosti”)

Primerjava: črna vs. bela škatla (2/2)

- **Potrebni sta obe metodi** testiranja
- Testiranje s črno in z belo škatlo sta oba končna ekstrema
- Vsaka izbira testnega primera leži **nekje vmes** in je odvisna od:
 - Števila možnih logičnih poti
 - Narave vhodnih podatkov
 - Množine računanja
 - Kompleksnosti algoritmov in podatkovnih struktur

Štirje koraki pri testiranju (1/2)

1. Izberemo, kaj želimo testirati

- Popolnost zahtev
- Testiranje kode za zanesljivost
- Testiranje učinkovitosti izvajanja

2. Odločimo se, kako bomo testirali

- Pregled (inšpekcija) kode
- Črna škatla, bela škatla
- Izbira strategije integracijskega testiranja
- ...

Štirje koraki pri testiranju (2/2)

3. Razvijemo testne primere

- Testni primer (test case) je množica testnih podatkov oz. situacij, ki jih bomo uporabili za izvajanje enote, ki jo testiramo oz. atributa, ki ga želimo izmeriti

4. Ustvarimo testni orakelj (test oracle)

- Testni orakelj vsebuje predvidene rezultate za množico testnih primerov
- Testni orakelj moramo definirati in zapisati, preden začnemo z dejanskim testiranjem

Izbira testnih primerov

- Uporabimo znanje, nabrano ob analizi zahtev, o funkcionalnih zahtevah (črna škatla):
 - Primeri uporabe (use cases)
 - Pričakovani vhodni podatki, neveljavni vhodni podatki
- Uporabimo znanje iz faze načrtovanja o programski strukturi (bela škatla):
 - Kontrolne strukture
 - Testiranje vejitev, zank, ...
 - Podatkovne strukture
 - Testiranje posameznih atributov v zapisih, polj, ...
- Uporabimo znanje iz faze implementacije o algoritmih:
 - Prisilno deljenje z nič
 - Uporabimo zaporedje testnih primerov za strežnik prekinitev

Testiranje učinkovitosti (1/2)

- Stresno testiranje (stress testing)
 - Skrajne limite sistema (maksimalno število uporabnikov, najzahtevnejša povpraševanja, ...)
- Količinsko testiranje (volume testing)
 - Testiramo, kaj se zgodi pri obravnavi ogromnih količin podatkov
- Konfiguracijsko testiranje (configuration testing)
 - Testiranje različnih konfiguracij strojne in programske opreme
- Testiranje kompatibilnosti (compatibility testing)
 - Testiranje kompatibilnosti z obstoječimi verzijami sistema
- Testiranje varnosti (security testing)
 - Poskušanje kršenja varnostnih zahtev

Testiranje učinkovitosti (2/2)

- Testiranje odzivnosti (timing testing)
 - Ovrednotenje odzivnih časov in časa, potrebnega za izvedbo posameznih funkcij
- Testiranje okolja (environmental test)
 - Testiranje tolerance na vročino, vlažnost, ...
- Testiranje kakovosti (quality testing)
 - Testiranje zanesljivosti, enostavnosti vzdrževanja, razpoložljivosti sistema
- Testiranje okrevanja (recovery testing)
 - Testiranje obnašanja sistema ob prisotnosti napak ali izgube podatkov
- Testiranje človeških faktorjev (human factors testing)
 - Testiranje uporabniških vmesnikov z uporabniki

Testiranje sprejemljivosti

▣ Alfa testiranje

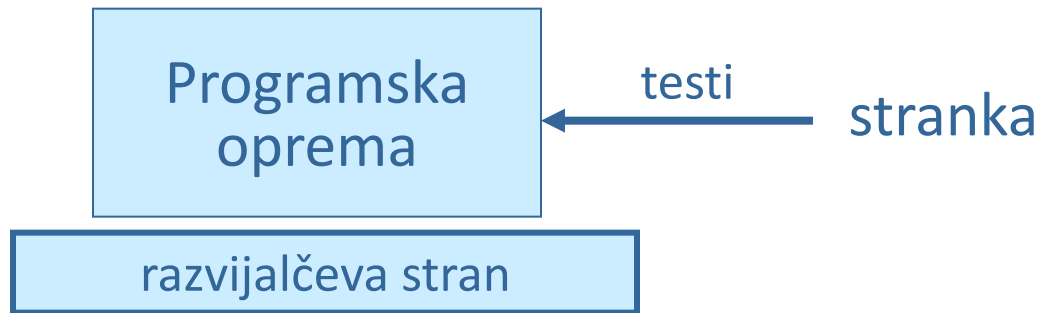
- ▣ Naročnik uporablja sistem pri razvijalcu
- ▣ Programska oprema se uporablja v nadzorovani postavitvi; razvijalec je zmeraj pripravljen odpraviti napake

▣ Beta testiranje

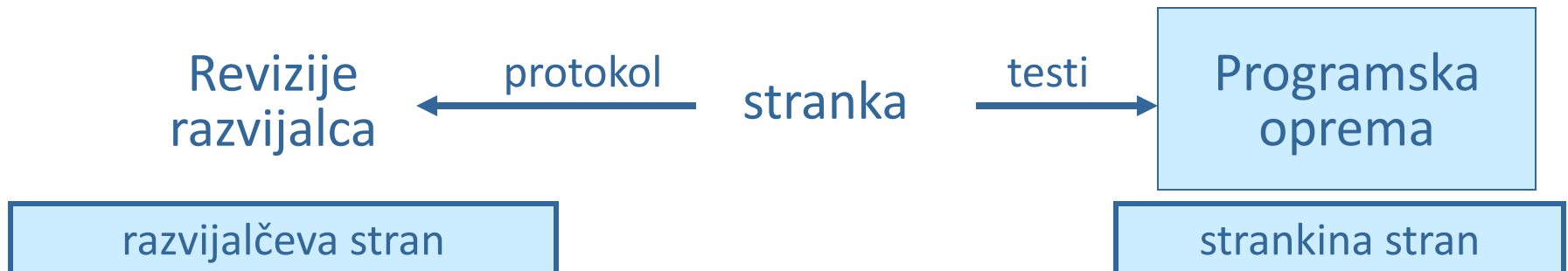
- ▣ Izvaja se pri naročniku, razvijalec ni prisoten
- ▣ Na sistemu se opravlja realistično preskušanje v ciljnem okolju
- ▣ Potencialna stranka lahko izgubi zaupanje

Alfa in beta testiranje

Alfa test

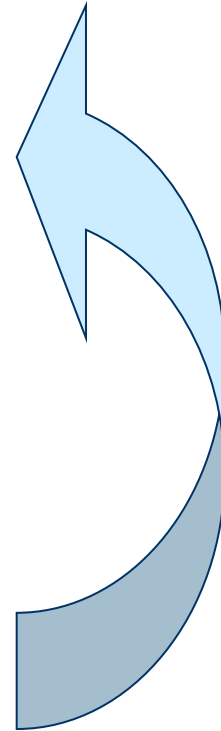


Beta test



Življenjski cikel testiranja

- Postavitev ciljev testiranja
- Načrtovanje testnih primerov
- Pisanje testnih primerov
- Testiranje testnih primerov
- Izvajanje testov
- Ovrednotenje rezultatov testiranja
- Spreminjanje sistema
- Izvajanje regresijskega testiranja
 - Regresijsko testiranje: ali smo s spremembo programa dodali kakšno novo napako v že testiran del sistema



Ekipo za testiranje

